

VaultDoor1

picoCTF Challenge Write-up

Category: Reverse Engineering | Difficulty: Easy

Challenge Details	
Challenge	VaultDoor1
Platform	picoCTF
Category	Reverse Engineering
Difficulty	Easy
Language	Java
Technique	Static analysis / index mapping

1. Challenge Overview

VaultDoor1 presents a compiled Java program that prompts the user for a vault password. The source code is provided, making this a static analysis task with no need to run or decompile anything.

The challenge description teases 'complicated arrays,' but inspection reveals a simple trick: the `checkPassword()` method validates each character of a 32-character password in a deliberately scrambled order rather than sequentially. The flag is effectively hiding in plain sight.

2. Source Code Analysis

2.1 Program Flow

The `main()` method strips the `picoCTF{}` wrapper from user input and passes the inner content to `checkPassword()`. If the method returns true, access is granted.

```
String input = userInput.substring("picoCTF{".length(), userInput.length()-1);
if (vaultDoor.checkPassword(input)) { ... }
```

2.2 The `checkPassword()` Method

The method returns a chain of boolean ANDs — one condition per character. Each condition compares a specific index position to a hardcoded character. The indexes are presented out of order, which is the only 'obfuscation' in play:

```
password.charAt(0) == 'd' &&
password.charAt(29) == '5' &&
password.charAt(4) == 'r' &&
password.charAt(2) == '5' &&
// ... 28 more conditions ...
```

Key observations:

- The password is exactly 32 characters (enforced by the first condition).
- Every index 0–31 is represented exactly once — no gaps, no duplicates.
- No hashing, encryption, or runtime obfuscation is applied.
- The full password is recoverable purely from source by sorting the conditions by index.

3. Solution

3.1 Approach

The solve is a one-step index sort. Parse each `charAt(n) == 'x'` pair, place the character at position `n` in a 32-element array, then read the array left-to-right.

3.2 Solver Script (Python)

```
chars = {}
chars[0] = 'd'; chars[1] = '3'; chars[2] = '5'; chars[3] = 'c'
chars[4] = 'r'; chars[5] = '4'; chars[6] = 'm'; chars[7] = 'b'
chars[8] = 'l'; chars[9] = '3'; chars[10] = '_'; chars[11] = 't'
chars[12] = 'H'; chars[13] = '3'; chars[14] = '_'; chars[15] = 'c'
chars[16] = 'H'; chars[17] = '4'; chars[18] = 'r'; chars[19] = '4'
chars[20] = 'c'; chars[21] = 'T'; chars[22] = '3'; chars[23] = 'r'
chars[24] = '5'; chars[25] = '_'; chars[26] = 'd'; chars[27] = '5'
chars[28] = '2'; chars[29] = '5'; chars[30] = '6'; chars[31] = '5'

password = ''.join(chars[i] for i in range(32))
print(f'picoCTF{{{password}}}')

```

3.3 Character Map (Sorted)

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
d	3	5	c	r	4	m	b	l
[9]	[10]	[11]	[12]	[13]	[14]	[15]	[16]	[17]
3	_	t	H	3	_	c	H	4
[18]	[19]	[20]	[21]	[22]	[23]	[24]	[25]	[26]
r	4	c	T	3	r	5	_	d
[27]	[28]	[29]	[30]	[31]				
5	2	5	6	5				

4. Flag

The reconstructed flag (obfuscated — solve the challenge to reveal it):

picoCTF{d35c***_****_d5****}**

Hint: the inner text is leet-speak for a two-word phrase meaning 'unscramble the characters.'

5. Key Takeaways

- Security through obscurity is not security. Shuffling array indices does not hide a secret — anyone with source access can sort them trivially.
- Static analysis is often sufficient. No dynamic execution, debugger, or decompiler was needed here.
- Secrets do not belong in source code. Even if obfuscated, constants embedded in code can be recovered. Proper solutions involve key derivation, secure storage, or challenge-response protocols.
- Leet-speak in flags is a telltale breadcrumb. The flag itself hints at the solve method: 'd35cr4mb13_th3_ch4r4cT3r5' = 'descramble the characters.'

6. References

- PicoCTF VaultDoor1 source: challenge-files.picoctf.net
- MITRE CWE-259: Use of Hard-coded Password
- MITRE CWE-656: Reliance on Security Through Obscurity